CHAPTER 09

Design A URL Shortener

Turn long URLs into short aliases (like tinyurl) and redirect billions of reads quickly.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a service like TinyURL: given a long URL return a much shorter one; when users open the short URL, redirect them to the original. Example: `https://www.example.com/q=chatsystem&c=loggedin` -> `https://tinyurl.com/y7keocwj`.

Why interviewers ask it: Classic warm-up: simple API, but touches hashing, ID generation, caching, redirect semantics, and read-heavy scale.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Traffic volume?	100 M URLs generated per day.
How short?	As short as possible.
Allowed characters?	[0-9, a-z, A-Z] = 62 characters.
Delete / update?	Not supported (simplifies design).
Availability/scale?	High availability, scalability, fault tolerance.

3. Requirements

Functional

- **Shorten:** long URL -> short URL.
- **Redirect:** short URL -> original (HTTP 301/302).

Non-functional

- Very high availability and low redirect latency.
- Unguessable-ish, collision-free short codes.
- Scales to billions of stored URLs; read-heavy.

4. Back-of-the-envelope estimation

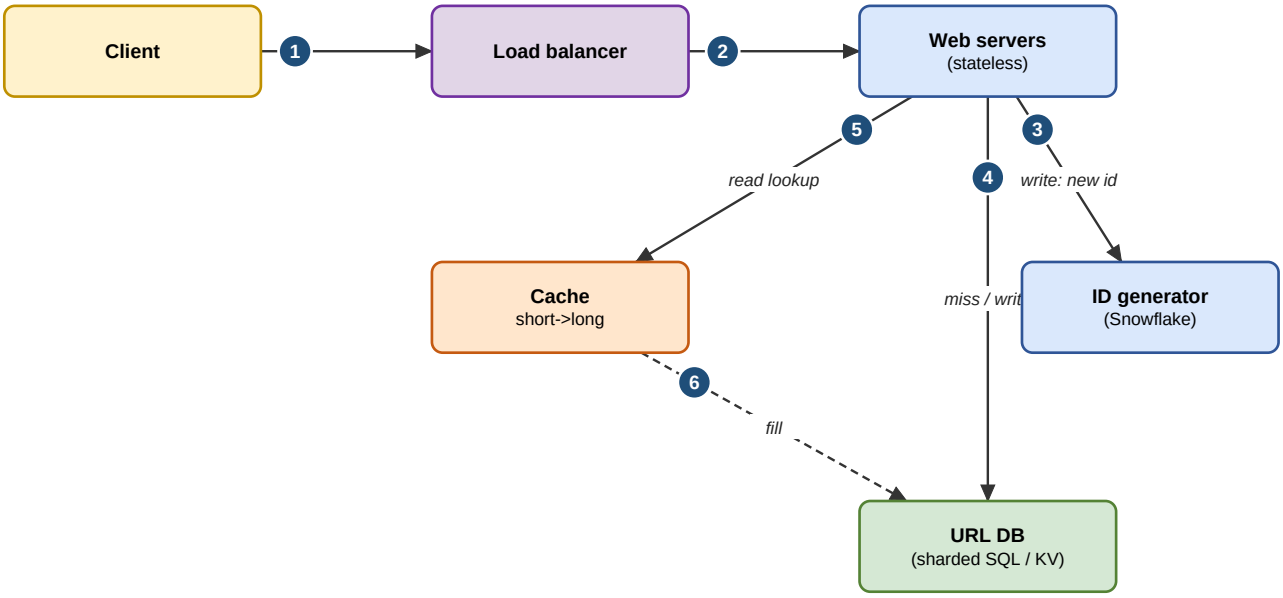
- Writes: 100 M/day / 86,400 = about **1,160 write QPS**. Reads with 10:1 ratio = about **11,600 read QPS** (peak 2x).
- Retention 10 years: 100 M x 365 x 10 = **365 B records**. At ~100 bytes each = about **36.5 TB**.
- Length of code: need 62^n >= 365 B -> n = 7 gives 62^7 = 3.5 trillion. So **7 characters**.

5. API / data model

```
POST /api/v1/data/shorten   body: { longUrl: 'https://...' }  -> 201 { shortUrl: 'https://tiny.co/zn9edcu' }
GET  /{shortUrl}            -> 301/302 Location: <longUrl>

Table url(id BIGINT PK, short_code CHAR(7) UNIQUE, long_url TEXT, created_at, expires_at)
```

6. High-level design



Step	What happens
1	**Shorten:** the client POSTs a long URL. The request goes through the load balancer.
2	A stateless web server validates the URL (format, blocklist) and checks whether it was already shortened.
3	It asks the **ID generator** (Snowflake or a leased counter range) for a unique numeric ID, then **Base62-encodes** it into a 7-character code.
4	It stores `(id, short_code, long_url, created_at)` in the URL DB and returns the short URL.
5	**Redirect:** on `GET /{code}` the server checks the cache first, because reads outnumber writes about 10 to 1.
6	On a cache miss it reads the DB and fills the cache. The server then answers **301** (cacheable) or **302** (to keep click analytics).

Request flow

- **Shorten:** validate long URL -> if already stored return existing -> else get unique ID -> Base62-encode to 7 chars -> store (id, code, longUrl) -> return short URL.
- **Redirect:** check cache by short code -> on miss read DB -> populate cache -> return 301/302 with `Location` header.
- **301 vs 302:** 301 (permanent) lets browsers cache, reducing server load; 302 (temporary) sends every click through you - needed for analytics.

7. Deep dive

Approach A: hash + collision resolution

- Apply CRC32 / MD5 / SHA-1 to the long URL and keep the first 7 chars of the hex/base62 value.
- Collision: check DB; if taken, append a predefined string to the URL and rehash. Use a **Bloom filter** to speed up existence checks.
- **Pros:** fixed length, no ID generator. **Cons:** DB lookup per write; collisions as data grows.

Approach B: Base62 conversion of a unique ID (preferred)

- Generate a globally unique numeric ID (Snowflake, chapter 08, or a distributed counter), then convert base 10 -> base 62 (0-9, a-z, A-Z).
- Example: 11157 (base 10) -> `2TX` (base 62). With a 64-bit ID the string is at most 11 chars; with sequential counters 7 chars covers 3.5 T.
- **Pros:** no collisions, no existence check. **Cons:** length grows with ID size; IDs are **predictable** (next code is guessable) - mitigate with ID shuffling/encryption or random counter ranges per server.

```
def base62(n):
    chars = '0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
    s = ''
    while n: n, r = divmod(n, 62); s = chars[r] + s
    return s or '0'
```

Deduplicating the same long URL

- Optional: keep a secondary index on `long_url` (hash) to return the existing short code. Many designs skip dedupe (each request gets a fresh code) to avoid the lookup.

Read path optimisation

- Reads dominate: front the DB with Redis/Memcached (LRU) - a small hot set (20%) serves most traffic; also use a CDN/edge for redirects.
- DB: shard by short code hash; replicas for reads. KV/NoSQL fits the pure key lookup access pattern.

Rate limiting and abuse

- Rate-limit creation per IP/user (chapter 05); validate and blacklist malicious domains (phishing, malware).

Analytics and expiry

- Log click events asynchronously to a queue (Kafka) for analytics. TTL / `expires_at` with a background cleanup job to reclaim codes.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Code generation	Hash + collision check / Base62 of unique ID	Base62 of ID avoids collisions; hash gives non-predictable codes.
Redirect status	301 permanent / 302 temporary	301 to cut load; 302 when you need click analytics or the ability to change targets.
Storage	Sharded SQL / NoSQL KV	KV scales simply for pure key lookups; SQL if you need rich queries/reporting.
Predictability	Sequential IDs / randomised IDs	Shuffle or encrypt IDs if enumeration is a concern.

9. Scaling, reliability, failure modes

- Stateless web tier behind LB; autoscale on QPS.
- Pre-generate ID ranges per server (e.g. each app server leases a block of 1,000 IDs) so no per-request coordination.
- Multi-region replication; DB failover; cache warm-up after restart.

10. Likely follow-up questions

- How do you avoid two servers generating the same code?
- How do you support custom aliases and expiry?
- How would you design click analytics?
- What happens at 100x traffic? (CDN edge redirect, bigger cache, more shards)

11. Common mistakes

- Choosing a 6-char code without checking capacity (62^6 = 56 B < 365 B).
- Ignoring 301 vs 302 semantics.
- Assuming the DB can serve all reads without a cache.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	4 min	"Let me confirm the numbers: roughly 100 M new URLs per day, characters limited to 0-9, a-z, A-Z, no delete or update. That gives about 1,160 writes per second and, with a 10:1 ratio, about 11,600 reads per second. Over 10 years that is 365 billion records, so I need 7 characters since 62^7 is about 3.5 trillion."
2. High-level design	10 min	"Two APIs: POST to shorten and GET to redirect. Stateless web servers behind a load balancer, an ID generator, a cache and a sharded URL store. For redirects I'll use 301 to reduce load, or 302 if the business needs click analytics. Let me walk through the shorten flow and then the redirect flow."
3. Deep dive	25 min	"I considered hash plus collision resolution versus Base62 of a unique ID. Hashing needs a database check per write, so I prefer a unique ID converted to Base62, which has no collisions. The downside is predictability, which I can mitigate by shuffling IDs. For read scale I use an LRU cache and read replicas; for writes I shard by short code and lease ID ranges to each server."
4. Wrap up	4 min	"I'd add rate limiting on creation, malware and phishing checks, and an async pipeline for click analytics. Failure handling is replication plus cache warm-up; to scale 100x I'd move redirects to the CDN edge."

13. 60-second recap / practice checklist

- Two APIs (shorten, redirect); 7-char Base62 code; about 1 K write QPS, about 10 K read QPS; about 36 TB in 10 years.
- Unique ID -> Base62 (or hash with collision handling). Cache + sharded KV store.
- 301 vs 302, rate limiting, analytics via async queue.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• GeeksforGeeks: Design a URL shortener (TinyURL)
• RFC 7231: 301 vs 302 semantics
• Base62 encoding (Wikipedia)